

Report team 17
Home Assignment 3
Problem Statement

Objective: Replace the Aerospace Block 6DOF from Homework 1 with your own system of equations derived in the NED coordinate system, following the example of symbolic calculations in Problem 3 of Homework 2. Implement the equations in Simulink and compare them with the Aerospace Block using the same test cases for degrees of freedom.

Steps:

- 1.1. Deriving the Equations: Derive your own system of equations in the NED coordinate system following the example of the symbolic calculations performed in Assignment 3 of Homework 2.
- 1.2. Implementation in Simulink: Implement the derived system of equations using basic blocks/functions in the Simulink environment.
- 1.3. Comparison of Results: Run the same test cases for degrees of freedom (from Assignment 1) through your own system and through the standard Aerospace Block 6DOF. Compare whether the results match.

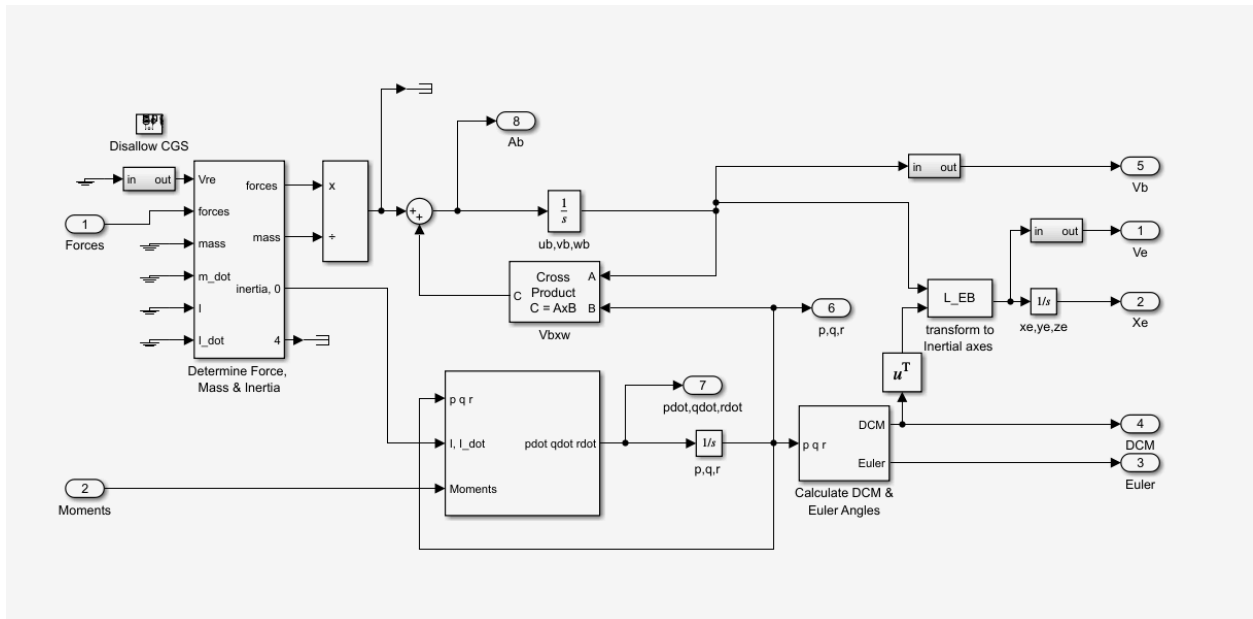


Figure 1. Expanded view on reference 6DOF.

1.1 Derivation of the 6DOF with Euler angles

% 6 Degree of Freedom (6DOF) with Euler's angles

% symbolic notations

syms mass real

% inertia tensor variables

```

syms lxx lyy lzz lxy lxz lyz real
syms u v w real
syms p q r real
syms Fx Fy Fz real
syms Mx My Mz real
syms phi theta psi real
% Velocity and Rate vectors
V_b = [u; v; w];
Omega = [p; q; r];
Forces = [Fx; Fy; Fz];
Moments = [Mx; My; Mz];
% Inertia tensor that assumes symmetry of the body
% as I12==I21, I13==I31, I23==I32
I = [lxx, lxy, lxz;
     lxy, lyy, lyz;
     lxz, lyz, lzz];
% 1. Accelerations
% from Second Newton's law
%  $F = m \cdot \dot{V}$ 
% Transport theorem
%  $\text{Vector}_{\text{inertial}} = \text{Vector}_{\text{local}} + \Omega \times \text{Vector}$ 
% combining those gives us
%  $F = m \cdot (\dot{V} + \Omega \times V_b)$ 
% solve for  $\dot{V}$ 
%  $\dot{V} = F/m - (\Omega \times V_b)$ 
V_dot = (Forces / mass) - cross(Omega, V_b);
u_dot = V_dot(1);
v_dot = V_dot(2);
w_dot = V_dot(3);
% 2. Body rotations rates
%  $M = \dot{H}$ 
%  $\dot{H} = I \cdot \dot{\Omega}$  - angular momentum
% and using Transport theorem
%  $\text{Moments} = I \cdot \dot{\Omega} + \Omega \times (I \cdot \Omega)$ 
% solve for  $\dot{\Omega}$ 
%  $\dot{\Omega} = \text{inv}(I) \cdot (\text{Moments} - \Omega \times (I \cdot \Omega))$ 
Omega_dot = inv(I) * (Moments - cross(Omega, I * Omega));
p_dot = Omega_dot(1);
q_dot = Omega_dot(2);
r_dot = Omega_dot(3);
% 3. Transformation of body rates to Euler angle rates
% Transform matrix for rotational rates
E = [ 1, 0, -sin(theta);
     0, cos(phi), sin(phi)*cos(theta);

```

```

    0, -sin(phi), cos(phi)*cos(theta)];
% Omega = E * Euler_rates
% therefore with E inverted we could do
% reversed transformation
Euler_rates = inv(E) * Omega;
phi_dot = Euler_rates(1);
theta_dot = Euler_rates(2);
psi_dot = Euler_rates(3);
% Earth-Relative Velocities
% The Direction Cosine Matrix (DCM) for standard Yaw-Pitch-Roll sequence
DCM = [cos(theta)*cos(psi), sin(phi)*sin(theta)*cos(psi) - cos(phi)*sin(psi),
cos(phi)*sin(theta)*cos(psi) + sin(phi)*sin(psi),
cos(theta)*sin(psi), sin(phi)*sin(theta)*sin(psi) + cos(phi)*cos(psi), cos(phi)*sin(theta)*sin(psi) -
sin(phi)*cos(psi),
-sin(theta), sin(phi)*cos(theta), cos(phi)*cos(theta)];
% Transforms body velocity to velocity in Earth frame of reference
V_e = DCM * V_b;
% Earth speed after transformation
V_ex = V_e(1);
V_ey = V_e(2);
V_ez = V_e(3);
disp('--- Inverse of the Euler Transformation Matrix (Euler to Body Rates) ---');
pretty(simplify(inv(E)))
disp('--- Body-Frame X-Axis Translational Acceleration (u_dot) ---');
pretty(simplify(u_dot))
disp('--- Body-Frame Y-Axis Translational Acceleration (v_dot) ---');
pretty(simplify(v_dot))
disp('--- Body-Frame Z-Axis Translational Acceleration (w_dot) ---');
pretty(simplify(w_dot))
disp('--- Body-Frame X-Axis Rotational Acceleration (p_dot) ---');
pretty(simplify(p_dot))
disp('--- Body-Frame Y-Axis Rotational Acceleration (q_dot) ---');
pretty(simplify(q_dot))
disp('--- Body-Frame Z-Axis Rotational Acceleration (r_dot) ---');
pretty(simplify(r_dot))
disp('--- Earth-Frame Roll Rate (phi_dot) ---');
pretty(simplify(phi_dot))
disp('--- Earth-Frame Pitch Rate (theta_dot) ---');
pretty(simplify(theta_dot))
disp('--- Earth-Frame Yaw Rate (psi_dot) ---');
pretty(simplify(psi_dot))
disp('--- Earth-Relative X-Axis Velocity (misnamed V_ex_dot) ---');
pretty(simplify(V_ex))
disp('--- Earth-Relative Y-Axis Velocity (misnamed V_ey_dot) ---');

```

```
pretty(simplify(V_ey))
disp('--- Earth-Relative Z-Axis Velocity (misnamed V_ez_dot) ---');
pretty(simplify(V_ez))
```

1.2. Implementation in Simulink

Implementation was done in such a way to resemble the existing 6DOF Module.

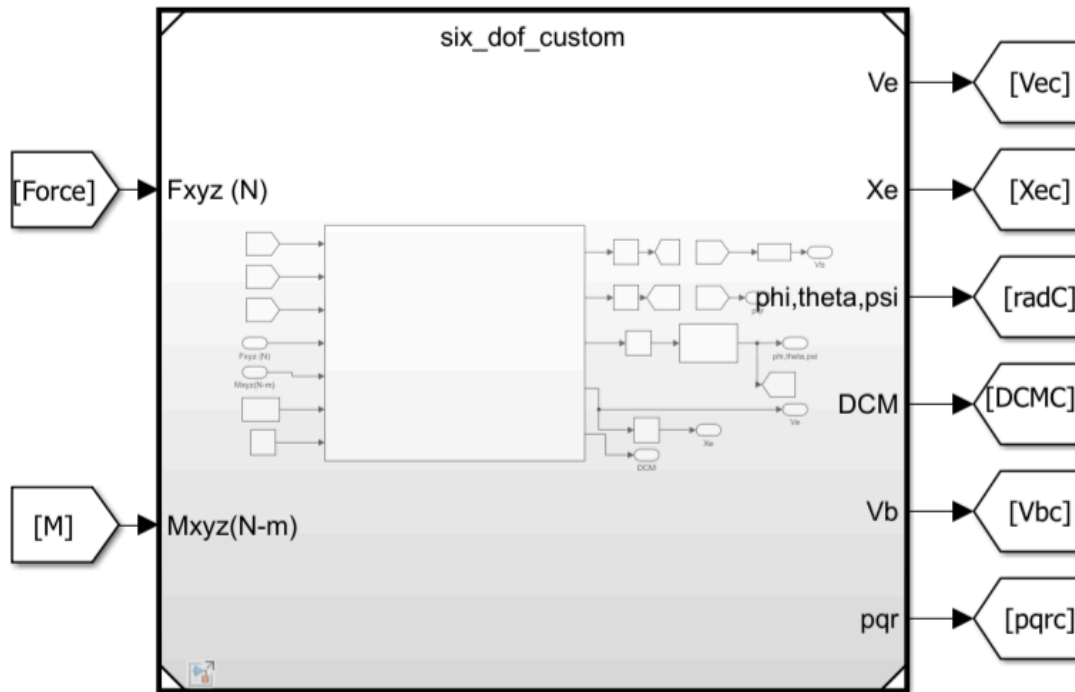


Figure 2. Custom 6DOF model.

Where:

INPUTS

Forces is a 3x1 vector representing the external body-frame forces (F_x , F_y , F_z).

M is a 3x1 vector representing the external body-frame torques (L , M , N).

OUTPUTS

Ve is a 3x1 vector representing the Earth-frame translational velocities ($\dot{x}$, $\dot{y}$, $\dot{z}$).

Xe is a 3x1 vector representing the Earth-frame spatial positions (x , y , z).

phi,theta,psi is a 3x1 vector representing the Earth-relative Euler angles for roll, pitch, and yaw.

DCM is a 3x3 matrix representing the Direction Cosine Matrix for the body-to-Earth coordinate transformation.

Vb is a 3x1 vector representing the body-frame translational velocities (u , v , w).

pqr is a 3x1 vector representing the body-frame angular velocities (p , q , r).

Internally it uses the MATLAB function block.

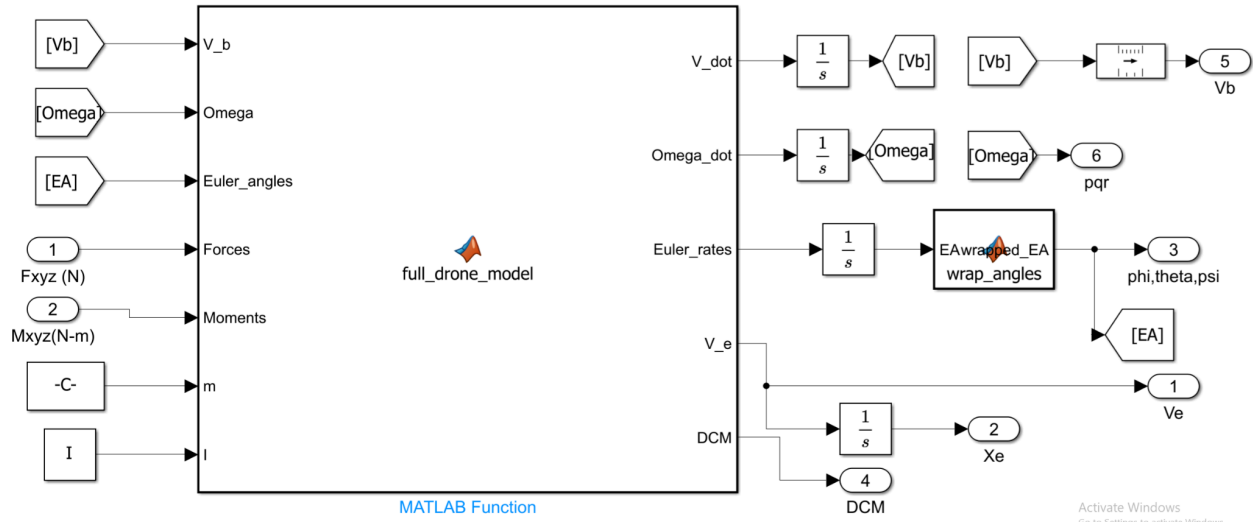


Figure 3. Custom 6DOF model - expanded.

Where:

INPUTS

V_b is a 3x1 vector representing the body-frame translational velocities (u, v, w).

Omega is a 3x1 vector representing the body-frame angular velocities (p, q, r).

Euler_angles is a 3x1 vector representing the Earth-relative angles (phi, theta, psi).

Forces is a 3x1 vector representing the external body-frame forces (F_x, F_y, F_z).

Moments is a 3x1 vector representing the external body-frame torques (L, M, N).

m is a scalar value representing the total mass of the drone.

I is a 3x3 matrix representing the inertia tensor.

OUTPUTS

V_{dot} is a 3x1 vector representing the body-frame translational accelerations (u_{dot}, v_{dot}, w_{dot}).

Omega_{dot} is a 3x1 vector representing the body-frame angular accelerations (p_{dot}, q_{dot}, r_{dot}).

Euler_rates is a 3x1 vector representing the Earth-frame angular rates (phi_{dot}, theta_{dot}, psi_{dot}).

V_e is a 3x1 vector representing the Earth-frame translational velocities (x_{dot}, y_{dot}, z_{dot}).

DCM is a 3x3 matrix representing the Direction Cosine Matrix for the body-to-Earth coordinate transformation.

Euler_rates wrapping block is necessary to correct the integrator's infinite accumulation of values, bounding the roll and yaw signals to the standard aerospace domain of **-pi, pi** radians to prevent continuous drift.

Wrapping block code is:

```
function wrapped_EA = wrap_angles(EA)
    % Ensure column vector
    EA = EA(:);
    wrapped_EA = zeros(3,1);

    % Wrap Roll, Pitch, Yaw to [-pi, pi]
    wrapped_EA(1) = mod(EA(1) + pi, 2*pi) - pi;
    wrapped_EA(2) = mod(EA(2) + pi, 2*pi) - pi;
    wrapped_EA(3) = mod(EA(3) + pi, 2*pi) - pi;
end
```

Code of the Custom 6DOF block.

```
function [V_dot, Omega_dot, Euler_rates, V_e, DCM] = six_dof_custom_block(V_b, Omega, Euler_angles, Forces, Moments, m, I)
    % Force all incoming signals to be column vectors

    % u,v,w
    % integrated V_dot
    V_b = V_b(:);
    % p,q,r
    % integrated Omega_dot
    Omega = Omega(:);
    % phi, theta, psi
    % integrated and wrapped Euler_rates
    Euler_angles = Euler_angles(:);

    % external inputs
    Forces = Forces(:);
    Moments = Moments(:);
    % Unpack angles
    phi = Euler_angles(1);
    theta = Euler_angles(2);
    psi = Euler_angles(3);
    % Body Accelerations
    %  $V_{\dot{}} = F/m - (\Omega \times V_b)$ 
    % u',v',w' acceleration body frame
    % integrate and will be velocity body frame
    V_dot = (Forces / m) - cross(Omega, V_b);
    % Angular Accelerations
    %  $\Omega_{\dot{}} = I^{-1} * (M - \Omega \times (I * \Omega))$ 
```

```

Omega_dot = inv(I) * (Moments - cross(Omega, I * Omega));
% Euler Angle Rates
E = [ 1, 0,      -sin(theta);
      0, cos(phi), sin(phi)*cos(theta);
      0, -sin(phi), cos(phi)*cos(theta)];

% phi', theta', psi'
Euler_rates = inv(E) * Omega;
% Earth-Relative Velocities
% The Direction Cosine Matrix (DCM) for standard Yaw-Pitch-Roll sequence
DCM = [cos(theta)*cos(psi), sin(phi)*sin(theta)*cos(psi) - cos(phi)*sin(psi),
cos(phi)*sin(theta)*cos(psi) + sin(phi)*sin(psi),
      cos(theta)*sin(psi), sin(phi)*sin(theta)*sin(psi) + cos(phi)*cos(psi),
cos(phi)*sin(theta)*sin(psi) - sin(phi)*cos(psi),
      -sin(theta),      sin(phi)*cos(theta),      cos(phi)*cos(theta)];

V_e = DCM * V_b;
end

```

1.3. Comparison of Results

Table 1. Commands and expected direction of movement in 6-DOF..

Rotors speed and command	Expected direction of movement	Explanation
====	+Z	With sufficient speed thrust generated by rotors will overcome gravity.
====, 0000, no command	-Z	Quadrotor will free fall from reached height.
+=== or ==-=	+X	Difference creates negative momentum around the Y axis. ($T_3 < T_1$)
==+= or -===	-X	Difference creates positive momentum around the Y axis. ($T_3 > T_1$)
+=== or ===-	+Y	Difference creates positive momentum around the X axis. ($T_2 > T_4$)

===+ or ===	-Y	Difference creates negative momentum around the X axis. ($T_2 < T_4$)
-------------	----	--

Simplest input. All motors with the same value.

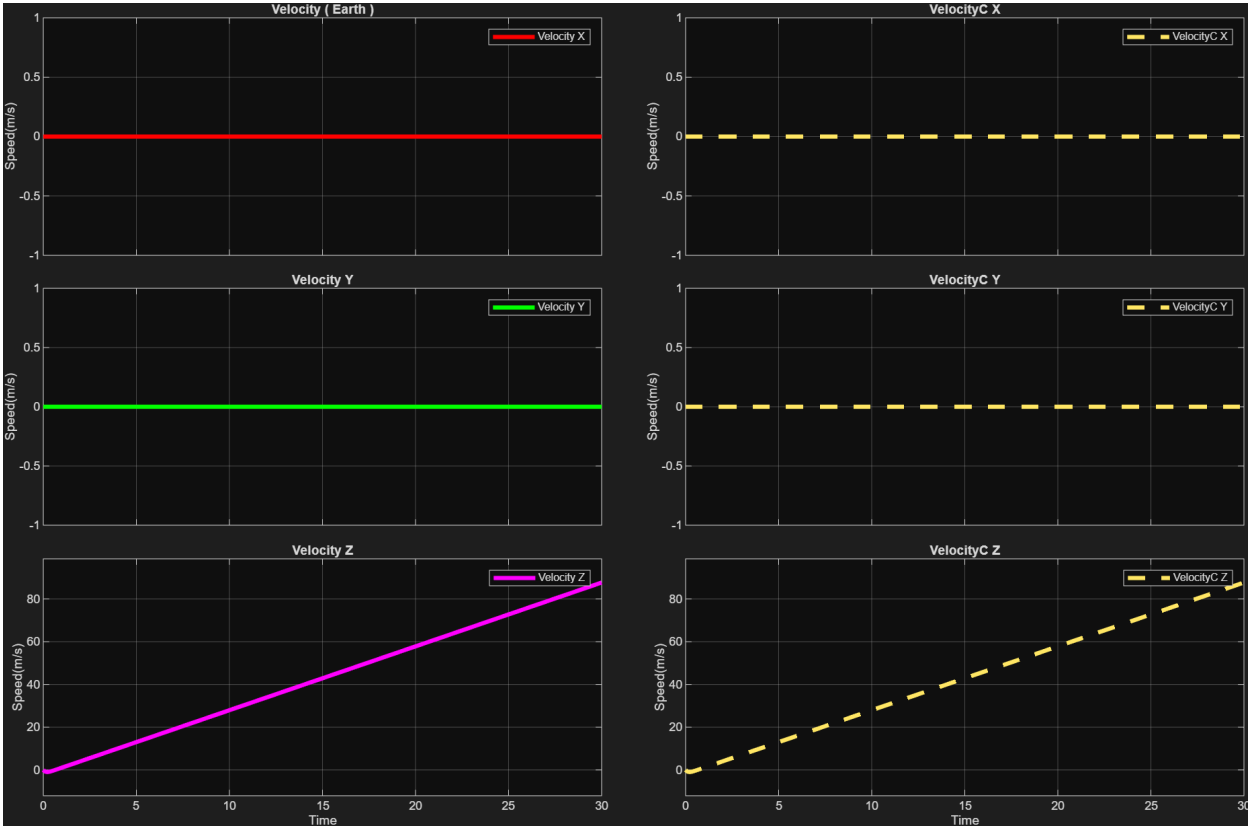


Figure 4. Velocity in Earth frame of reference - output from 6DOF subsystem. Motors values are 680.

On the **right** side are output values of existing reference implementation 6DOF and on the **left** custom build. This pattern of comparison will continue throughout the report.

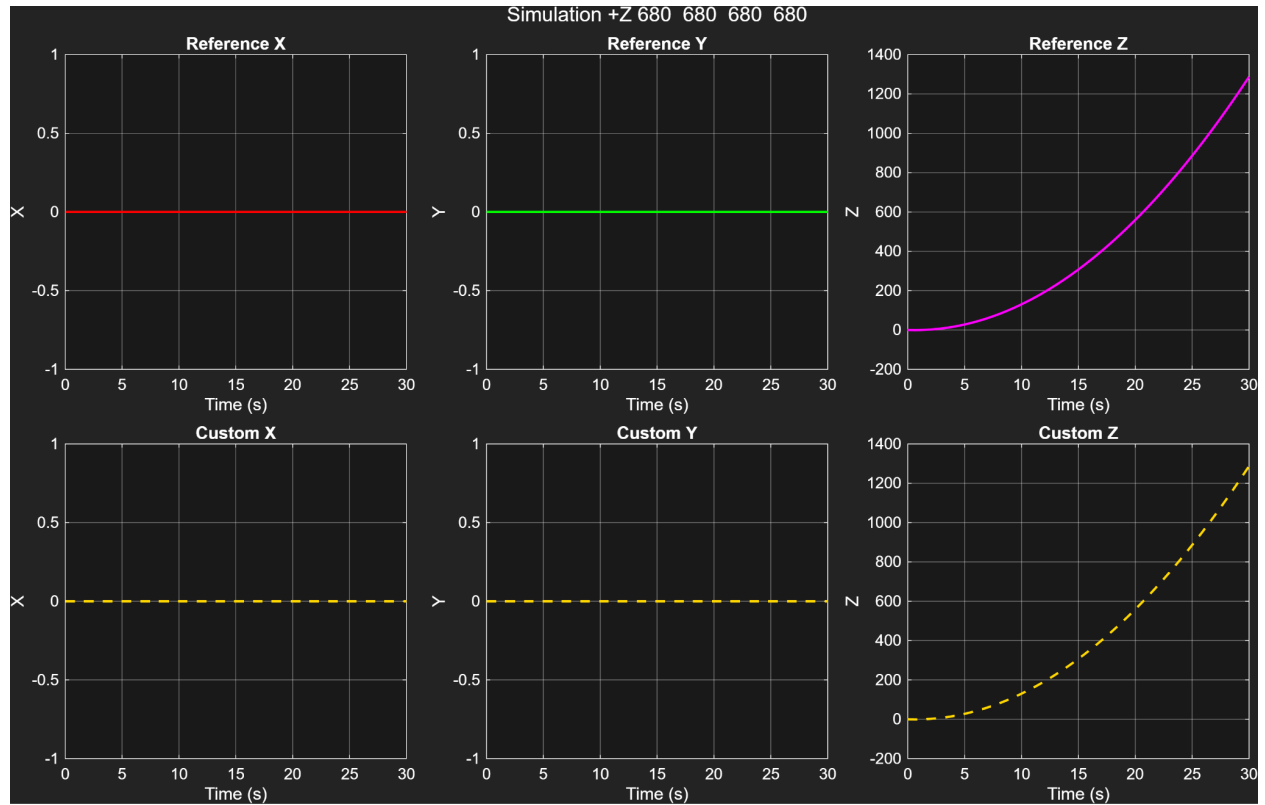


Figure 5. Position along X,Y,Z coordinates in Earth frame of reference - output from 6DOF subsystem. Motors values are 680. On the **right** side are output values of existing 6DOF and on the **left** custom build. This also corresponds to Direction +Z.

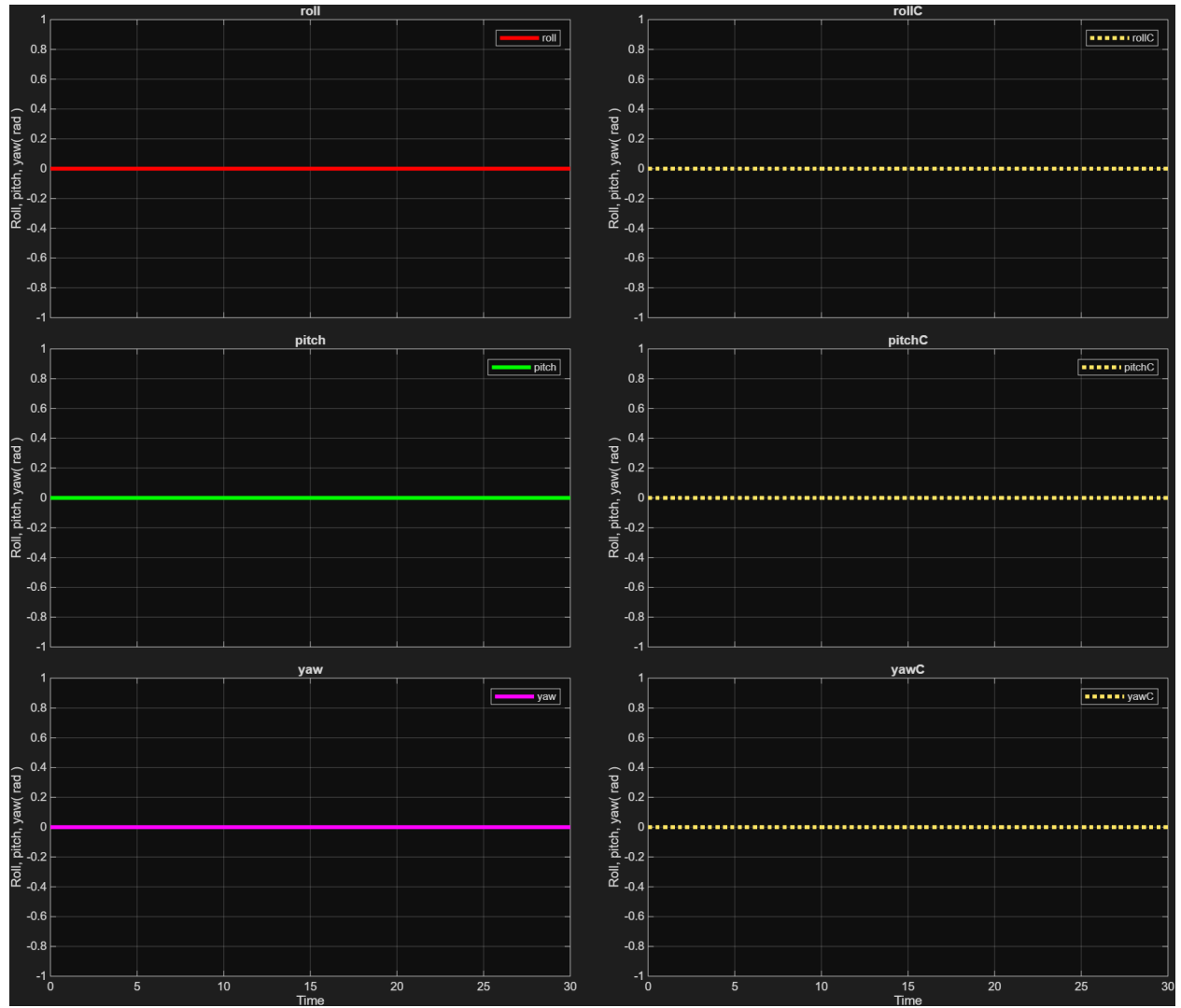


Figure 6. **Euler angles** - output from 6DOF subsystem. Motors values are 680. On the **right** side are output values of existing 6DOF and on the **left** custom build.

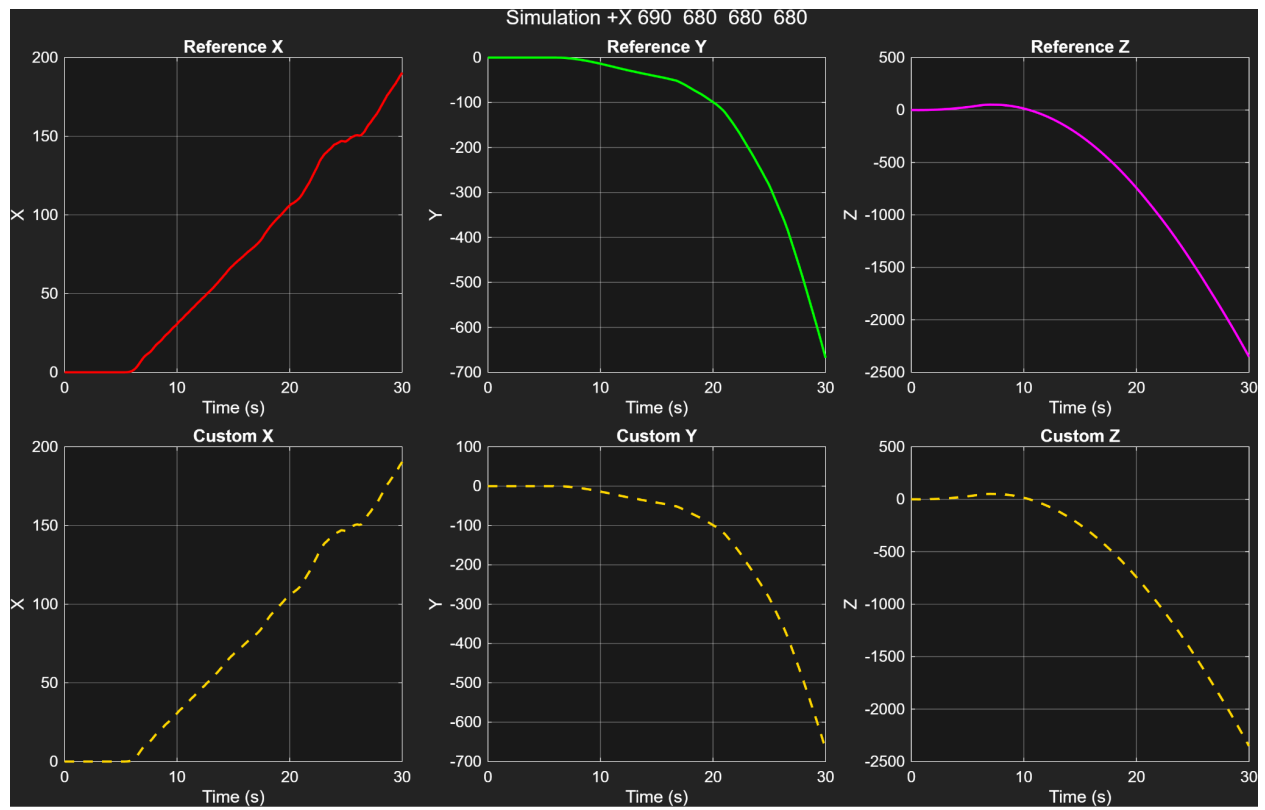


Figure 7. **Direction +X**. On the **top** part are output values of existing 6DOF and on the **bottom** custom build. **Speed set on rotors: 690 680 680 680 (+===)**

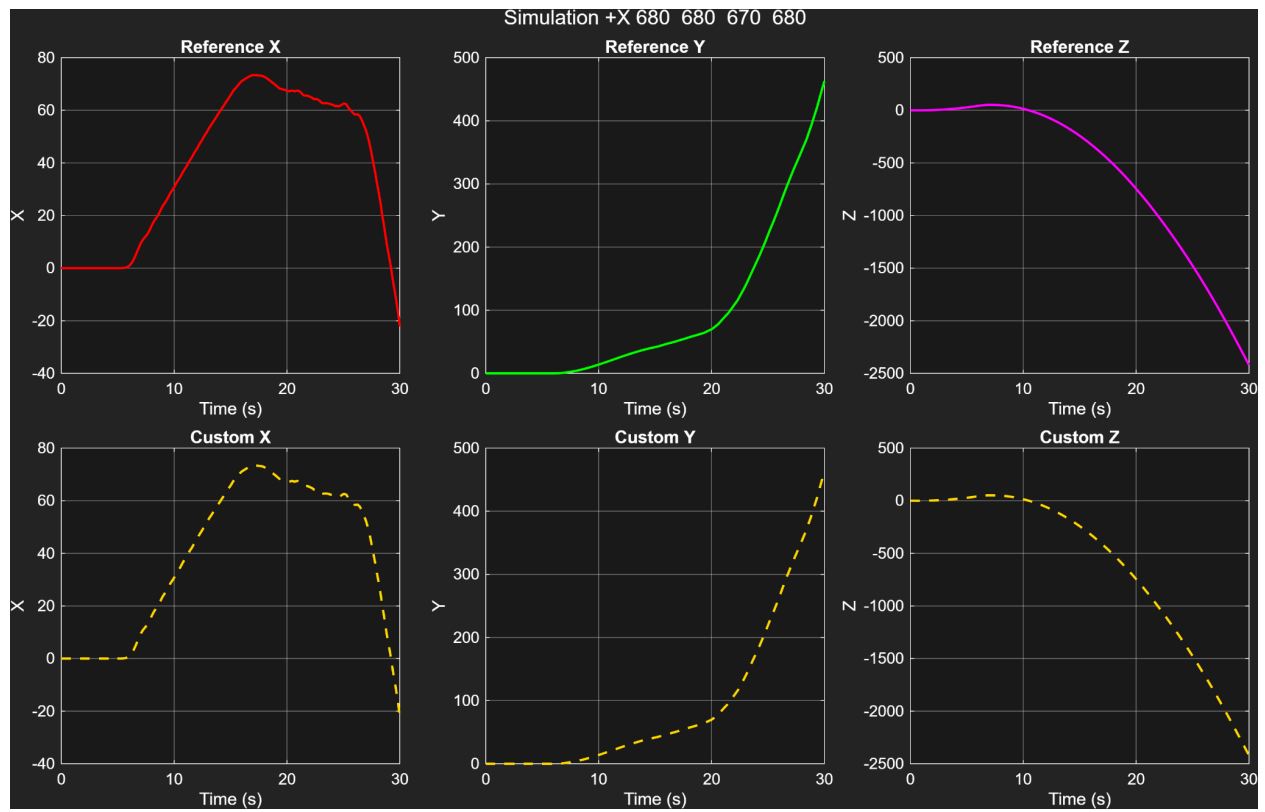


Figure 8. **Direction +X**. On the **top** part are output values of existing 6DOF and on the **bottom** custom build. **Speed set on rotors: 680 680 670 680 (==)**

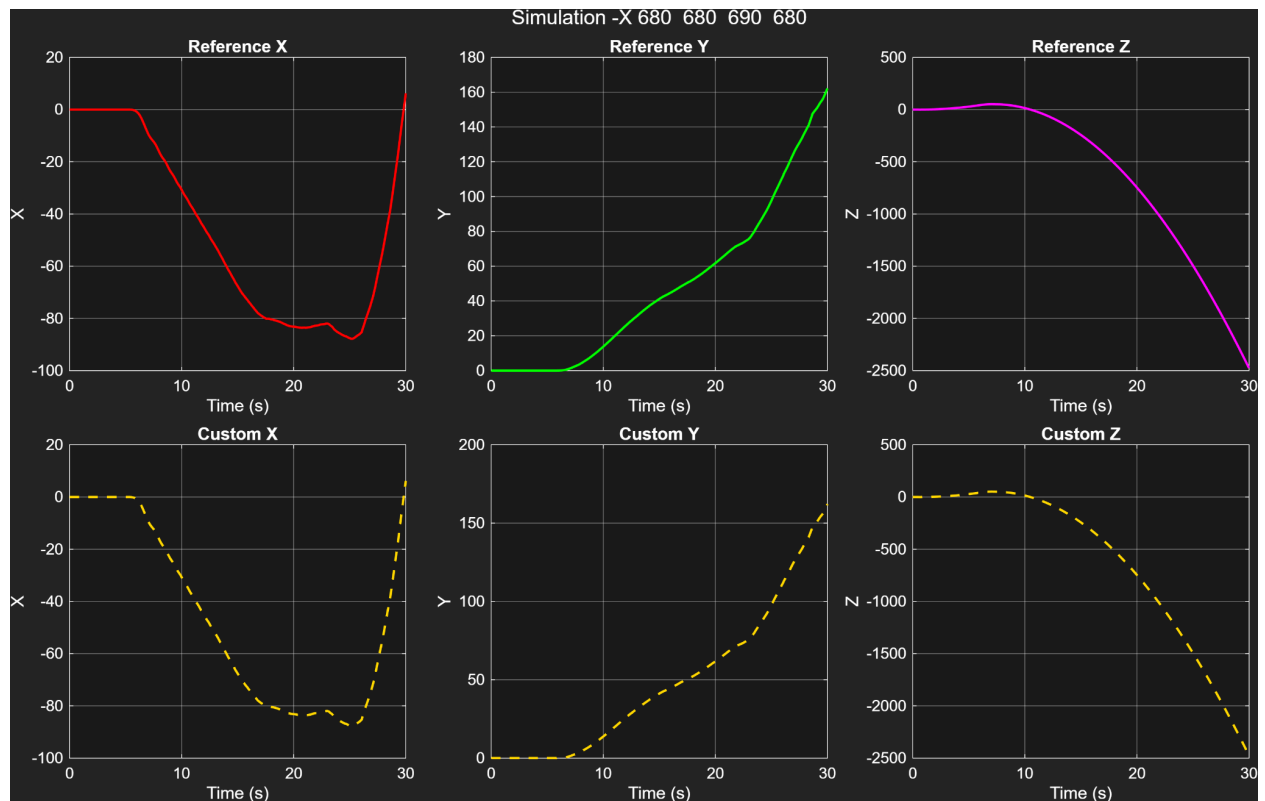


Figure 9. **Direction -X.** On the **top** part are output values of existing 6DOF and on the **bottom** custom build. **Speed set on rotors: 680 680 690 680 (==+=)**

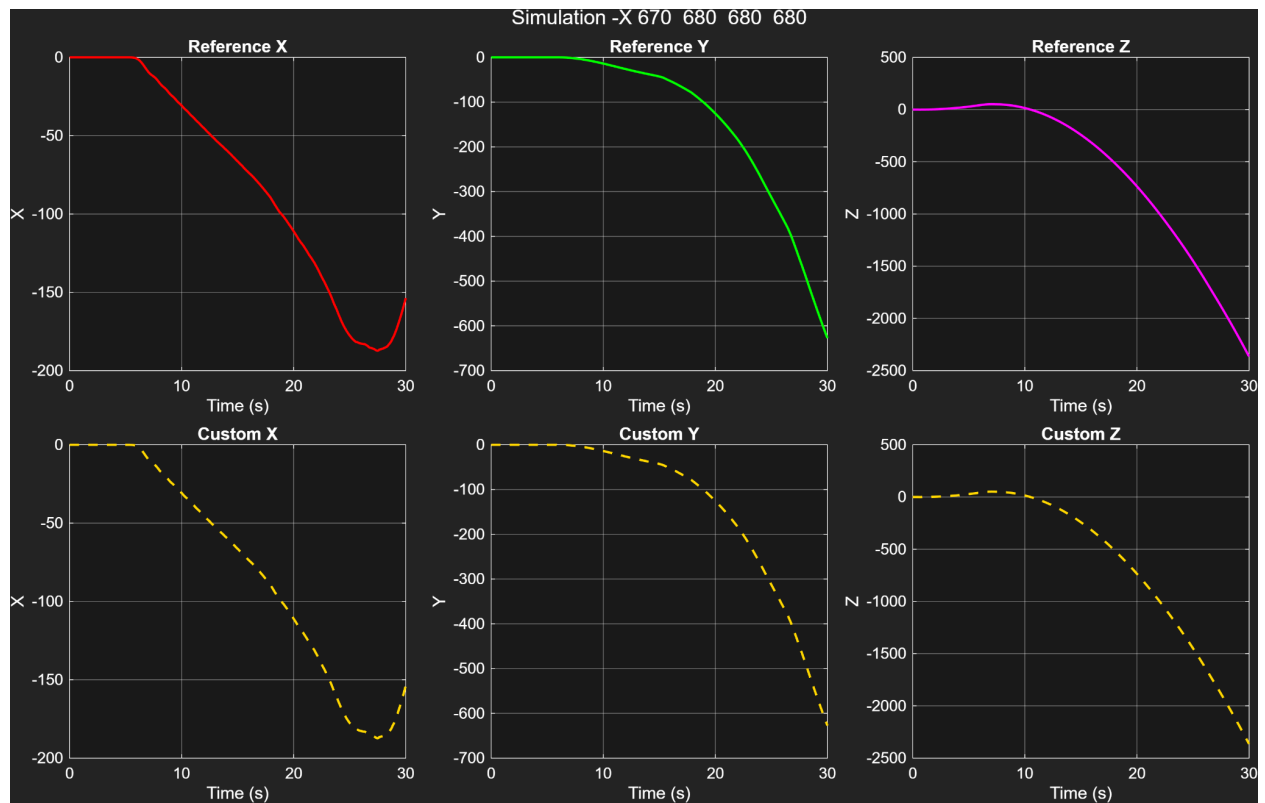


Figure 10. **Direction -X.** On the **top** part are output values of existing 6DOF and on the **bottom** custom build. **Speed set on rotors: 670 680 680 680 (-==)**

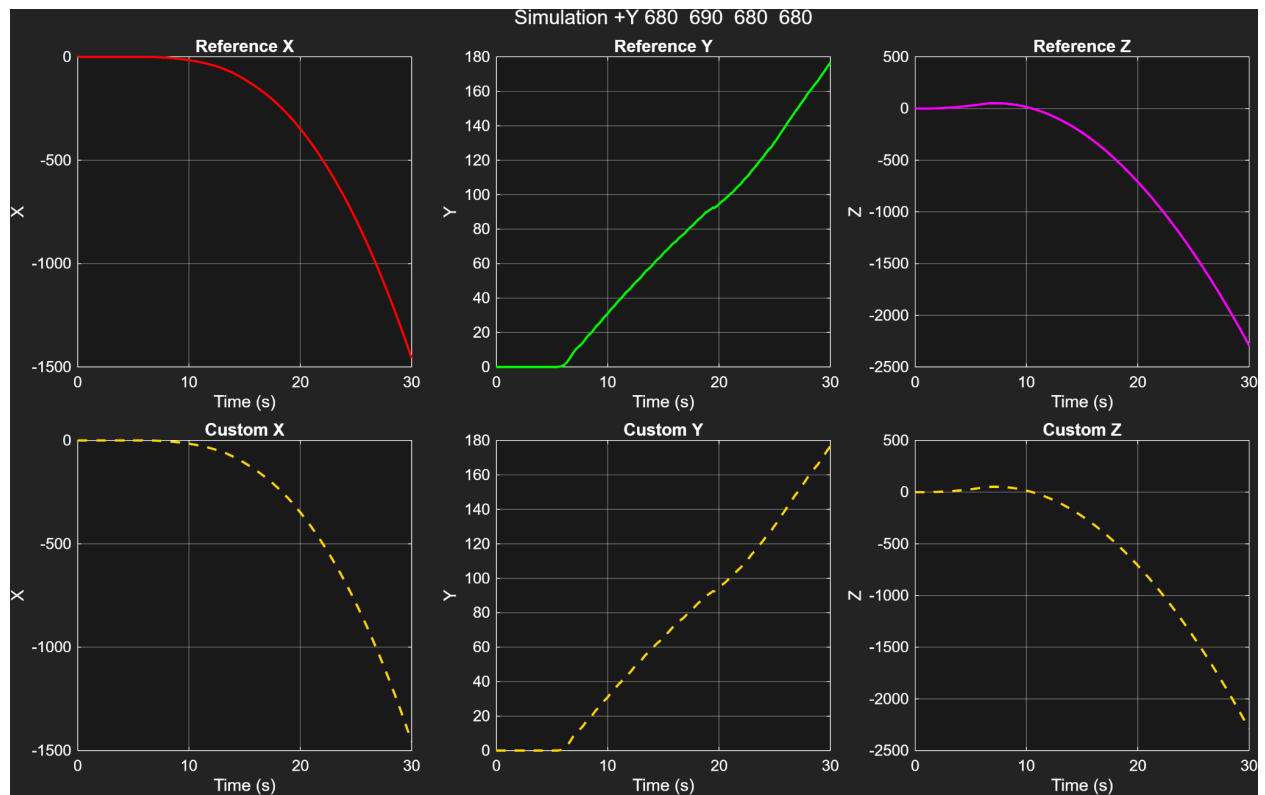


Figure 11. **Direction +Y.** On the **top** part are output values of existing 6DOF and on the **bottom** custom build. **Speed set on rotors: 680 690 680 680 (=+==)**

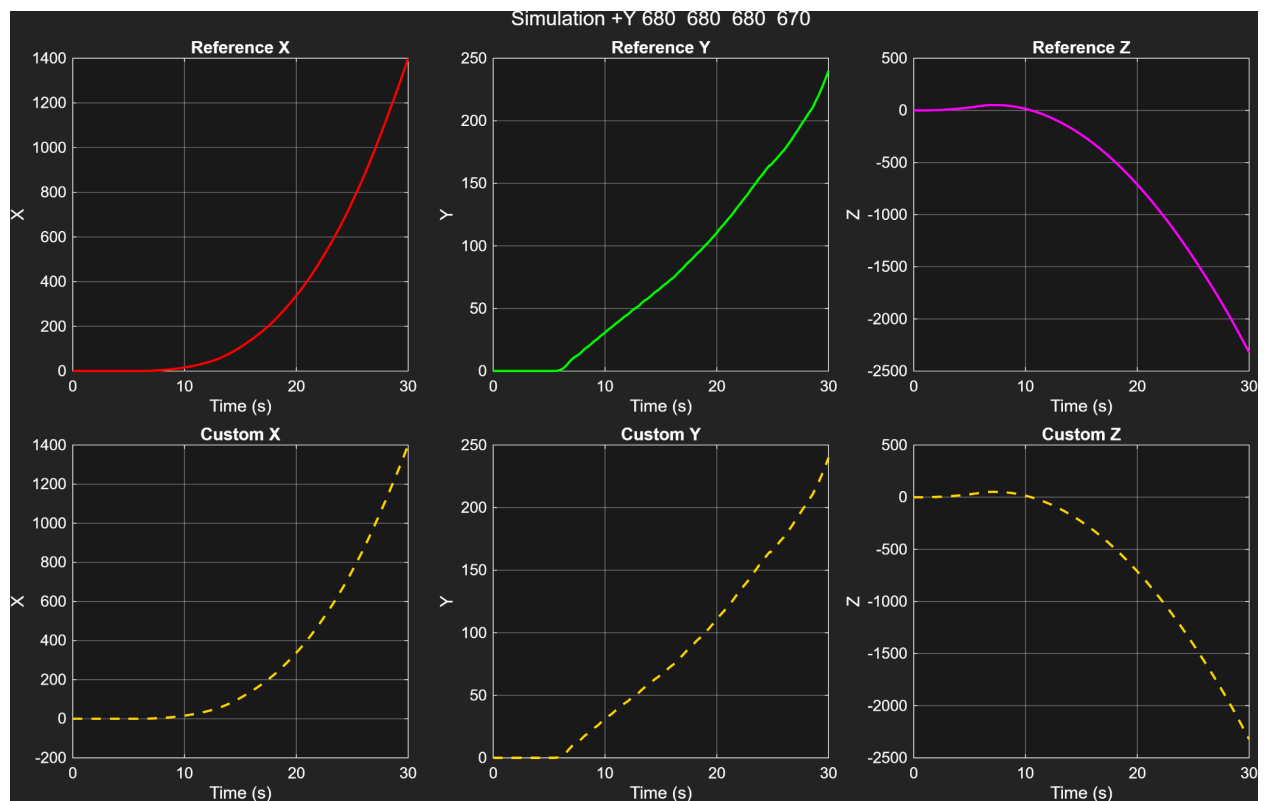


Figure 12. **Direction: +Y.** On the **top** part are output values of existing 6DOF and on the **bottom** custom build. **Speed set on rotors: 680 680 680 670 (===-)**

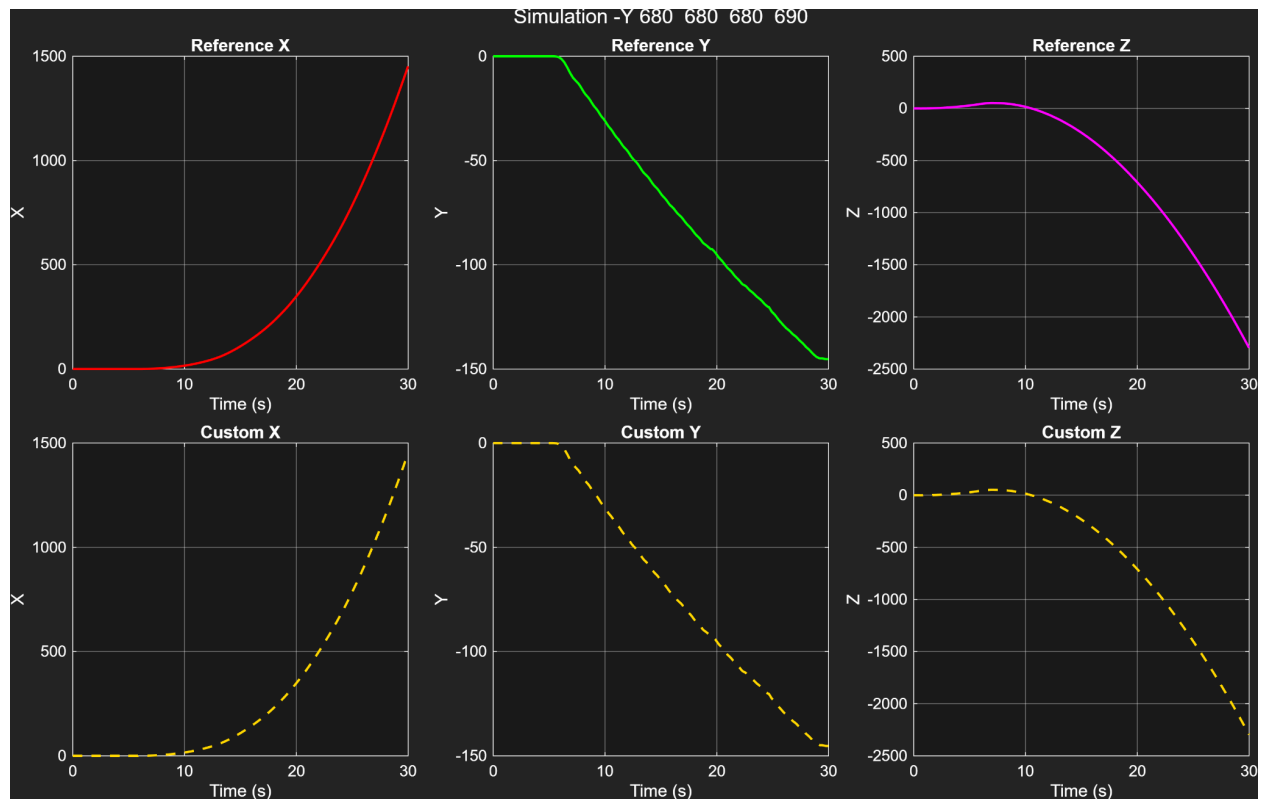


Figure 13. **Direction: -Y.** On the **top** part are output values of existing 6DOF and on the **bottom** custom build. **Speed set on rotors: 680 680 680 690 (===+)**

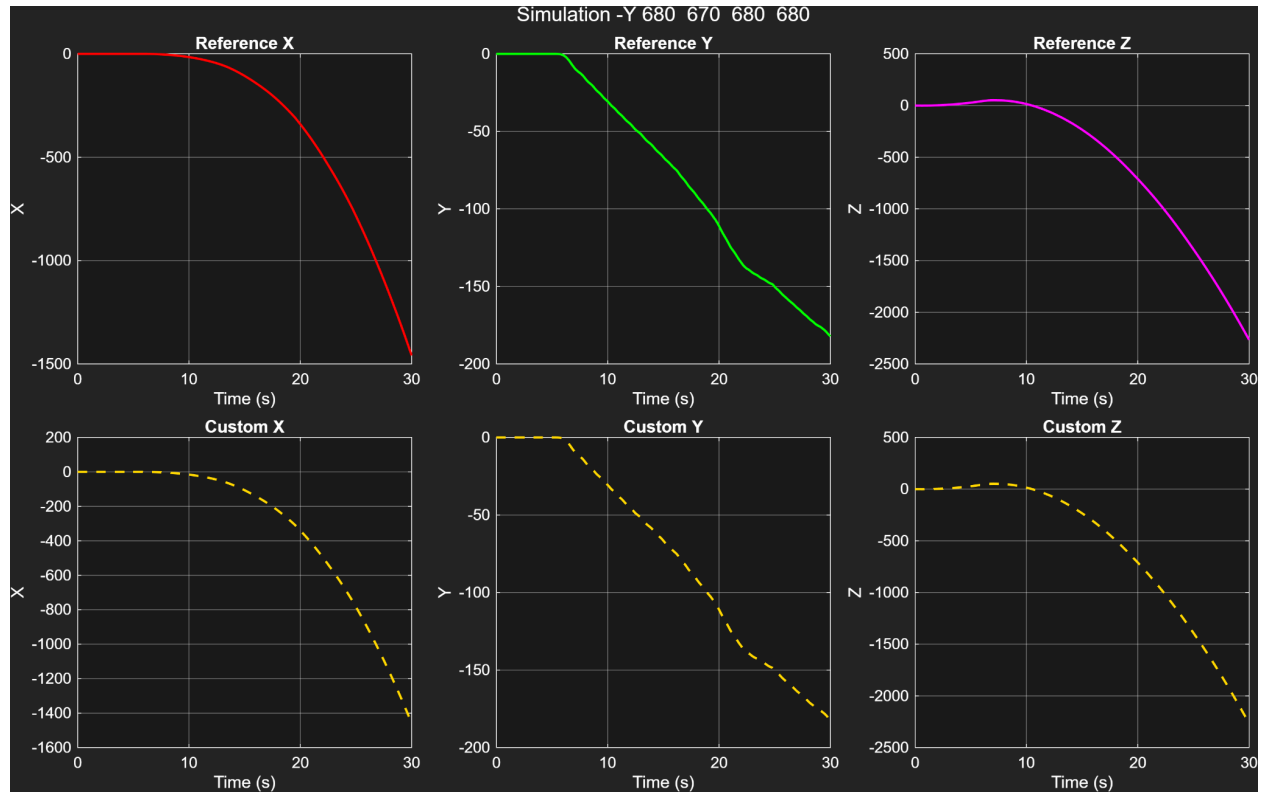


Figure 14. **Direction: -Y.** On the **top** part are output values of existing 6DOF and on the **bottom** custom build. **Speed set on rotors: 680 670 680 680 (=-==)**

Isolated roll

We would like to simulate roll change in a quadrotor. This is achieved by introducing imbalance between rotors speed 2 and 4. To have better isolated change in roll we would need to keep thrust almost the same and to achieve that we introduce step function (step time 4 seconds ; step value 10), which will simultaneously increase speed on one rotor but decrease to the same degree on another.

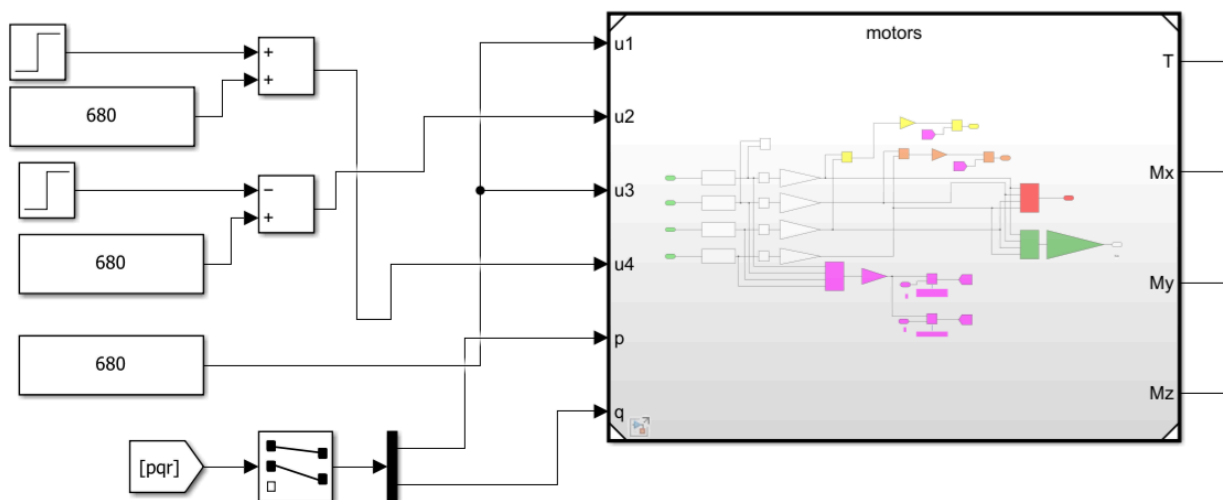


Figure 15. Isolated roll in simulink model.

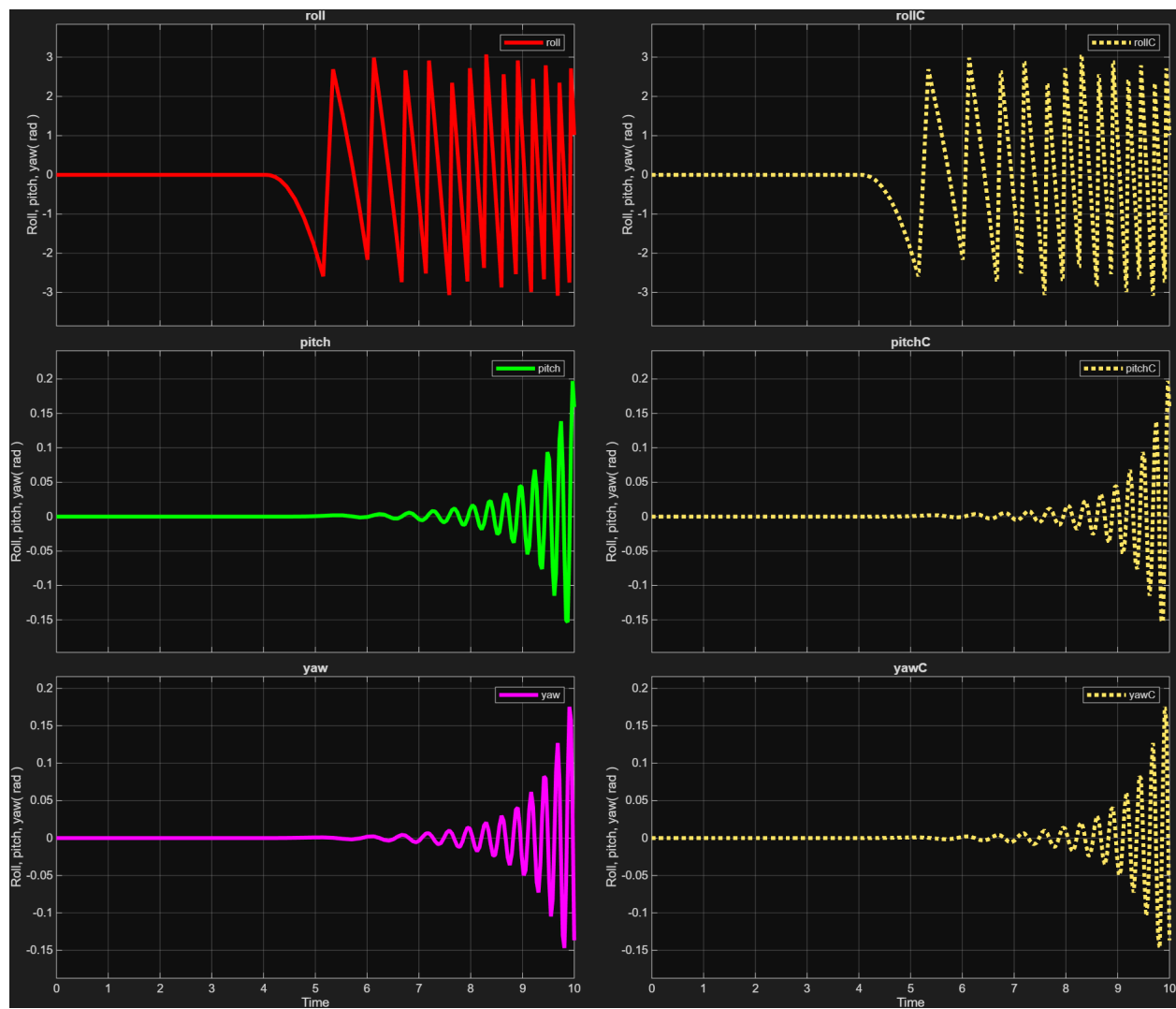


Figure 16. Isolated roll. When the bigger speed is on rotor 4.

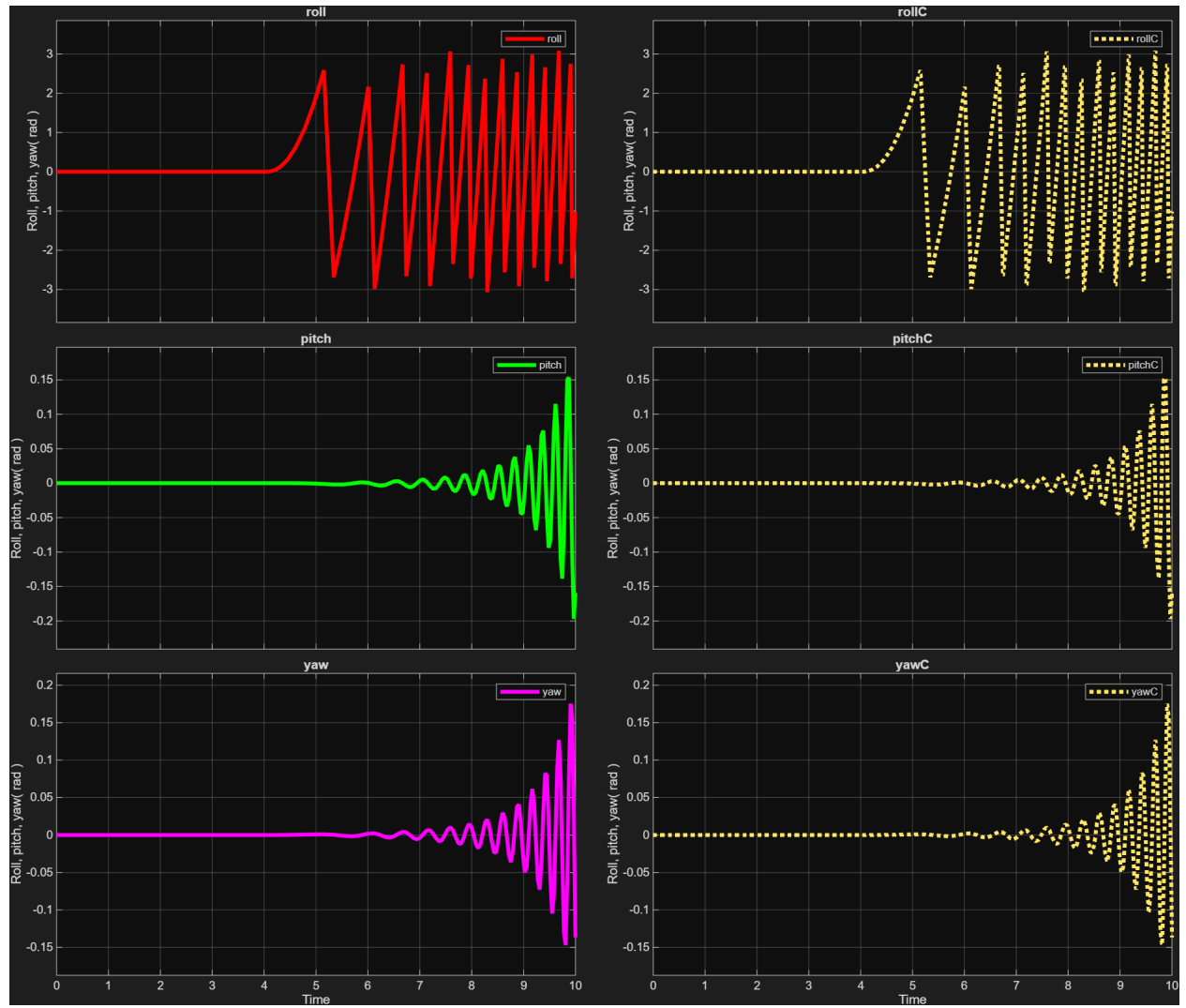


Figure 17. Isolated roll. When the bigger speed is on rotor 2.

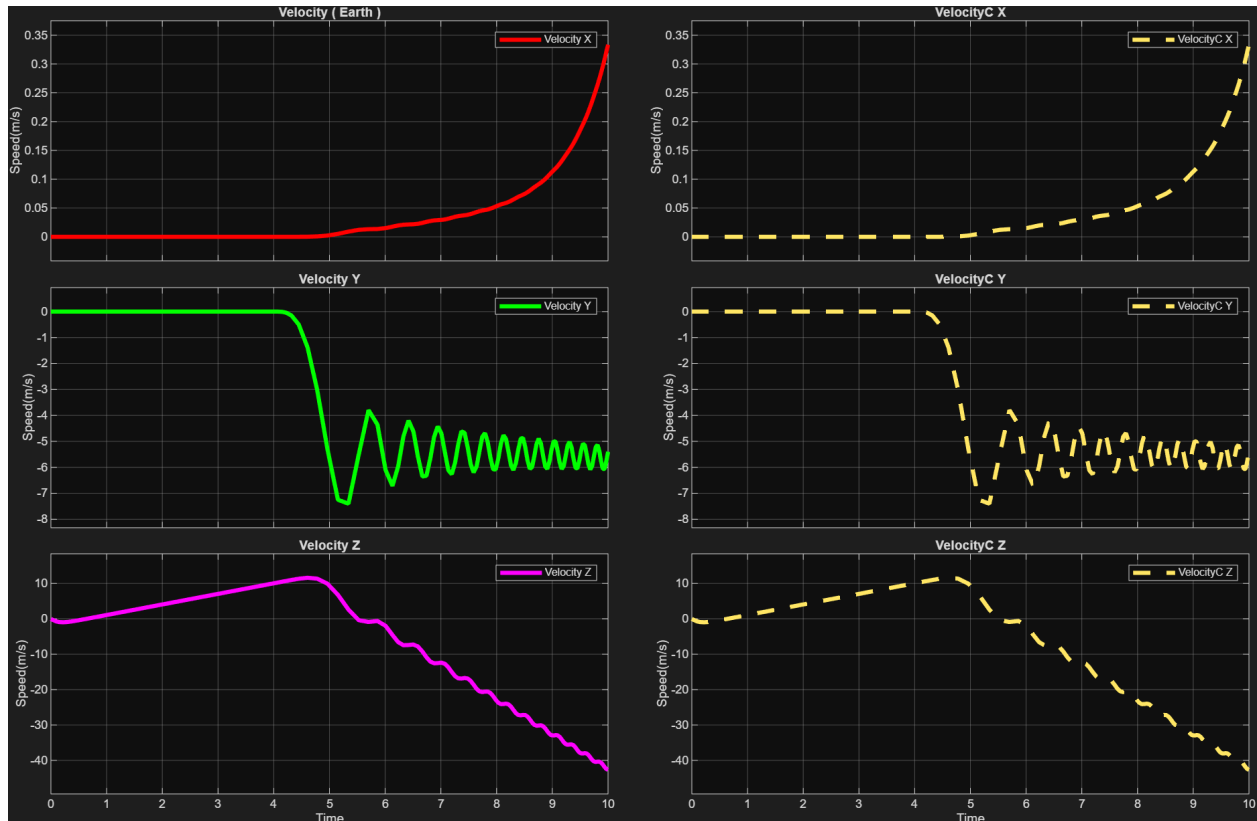


Figure 18. Velocity (Earth frame) during isolated roll.

Isolated pitch

Similar module as for isolated roll but motors that are changing are 1 and 3 (Step time 4 seconds ; step value 10).

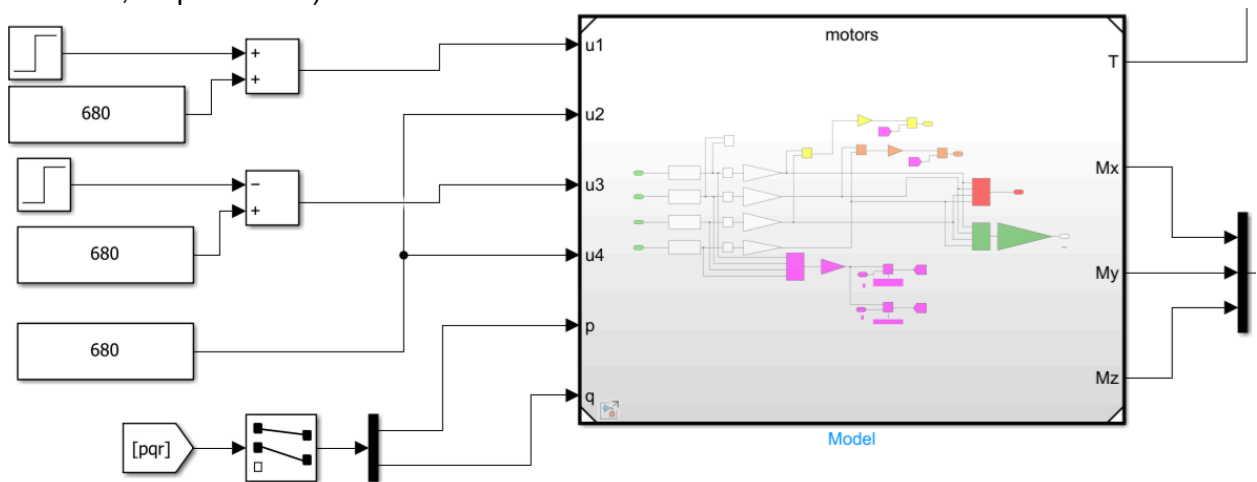


Figure 19. Isolated roll in simulink model.

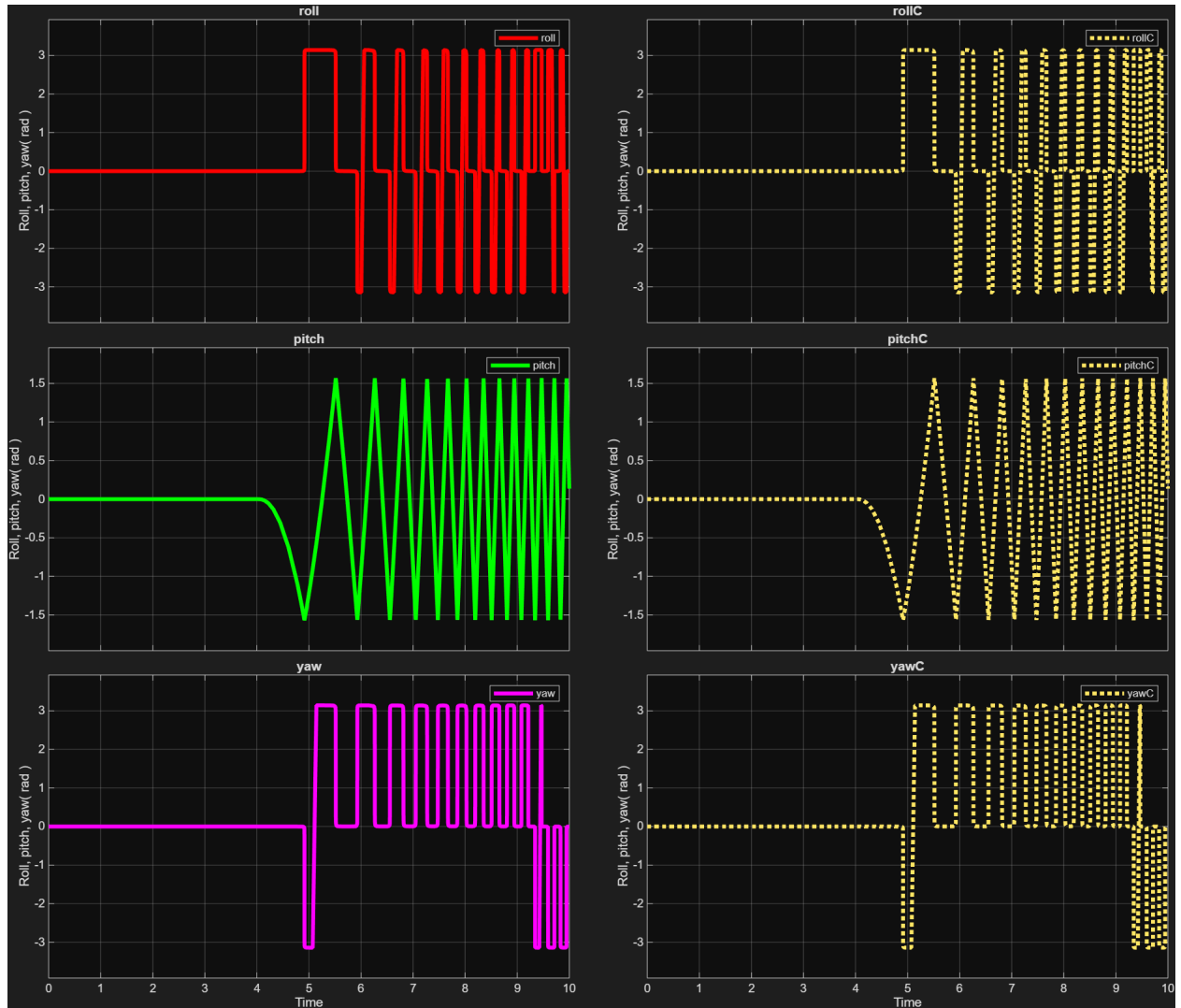


Figure 20. Isolated pitch. When the bigger speed is on rotor 1. Roll and Yaw flips due to **gimbal lock** effect which happens in the Euler angel system with pitch ± 90 degrees which causes the X axis to be parallel to Z and that in turn makes subsequent Roll or Yaws indistinguishable from each other (aka. lock) and in modeling it occurs as a sudden flips. To avoid this phenomenon, use of quaternions is advised.

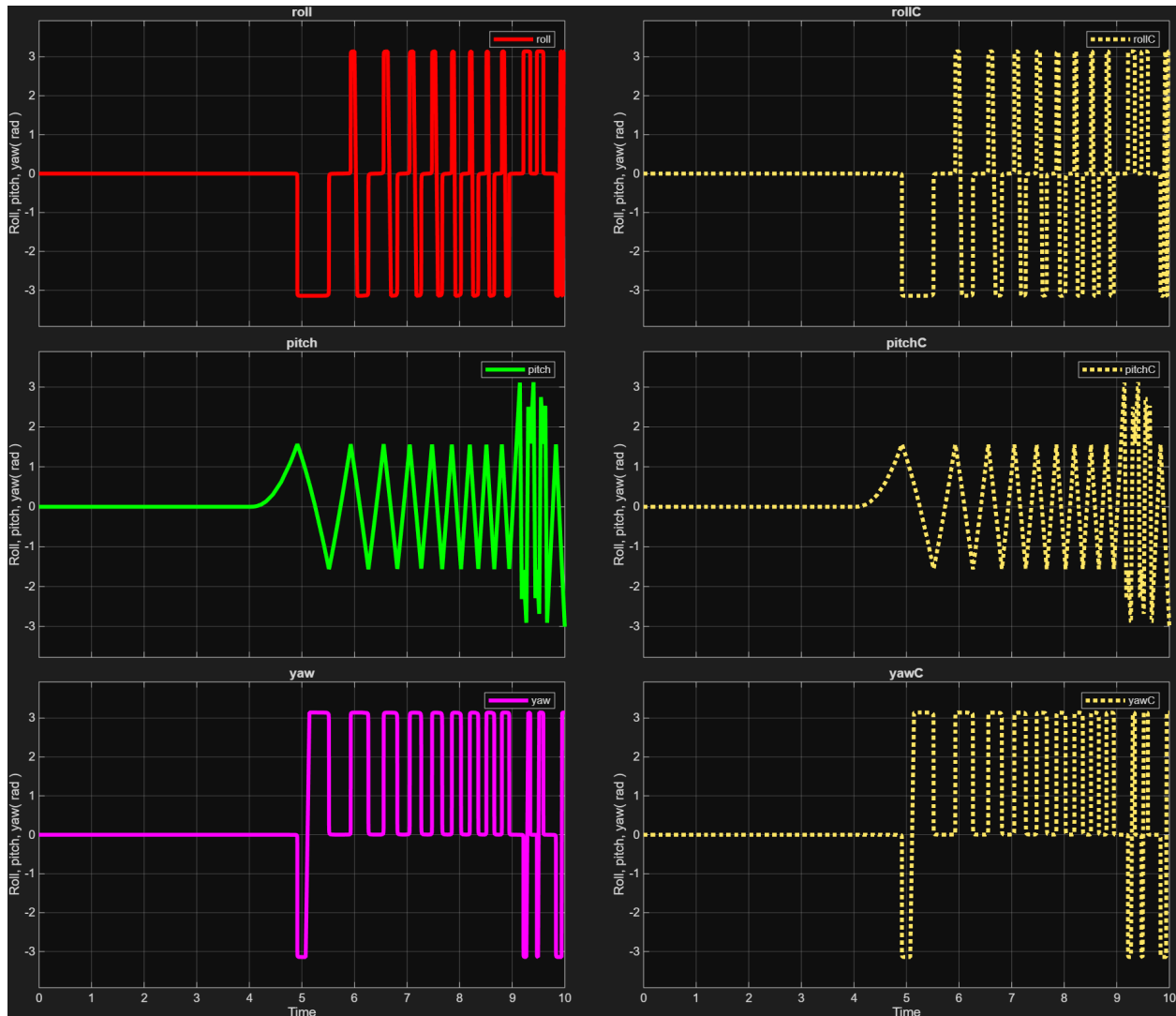


Figure 21. Isolated pitch. When the bigger speed is on rotor 3. With gimbal lock in different directions as well.

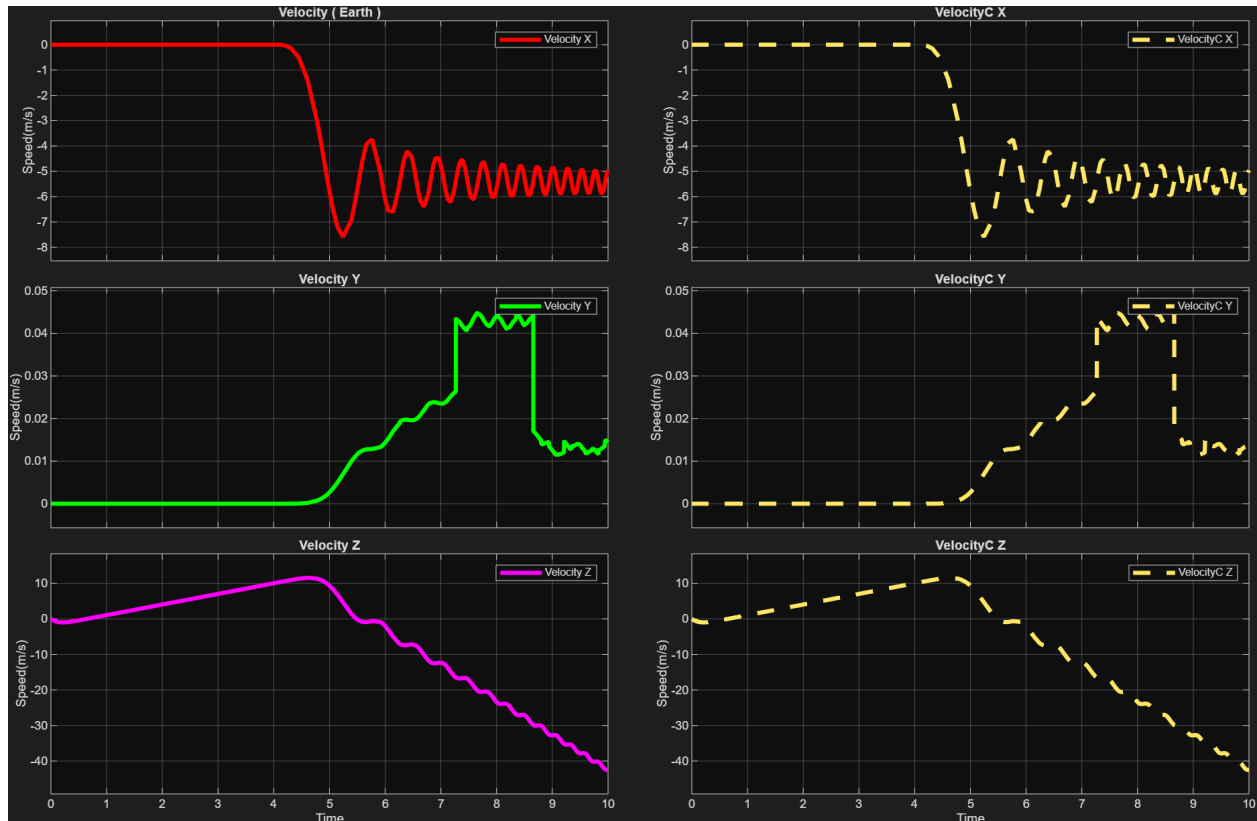


Figure 22. Velocity (Earth frame) during isolated pitch.

Isolated yaw

Yaw is achieved by 3rd Newton's law. In our system this means that either clock wise or counter clock wise rotors are spinning faster (or slower) this imbalance creates momentum in the opposite direction to the faster spinning pair.

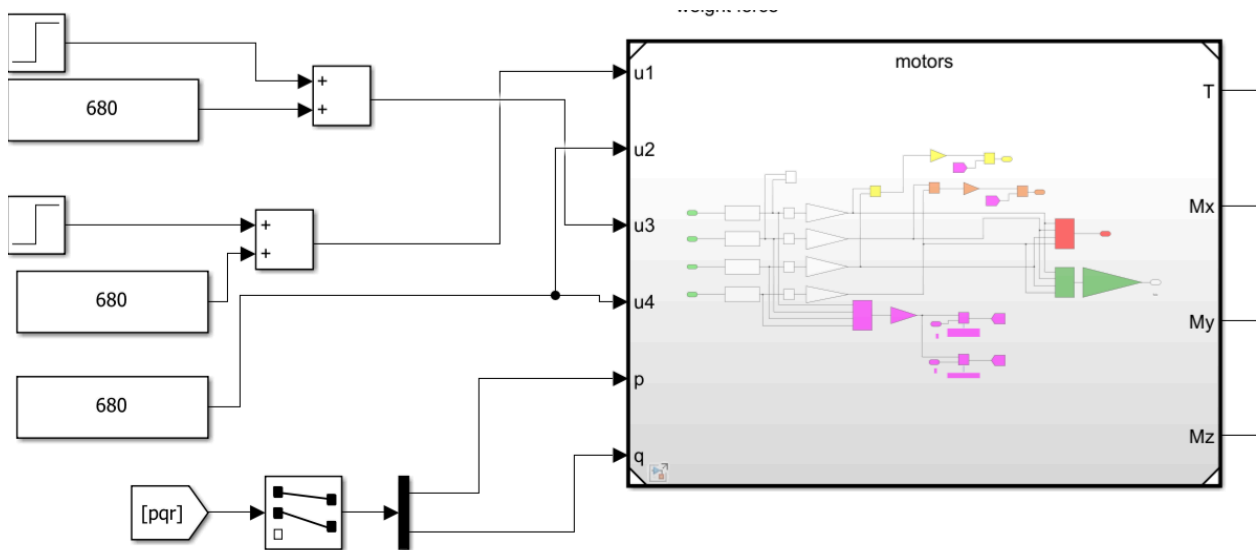


Figure 23. Isolated yaw in simulink model. We use rotors 1 and 3 to control yaw.

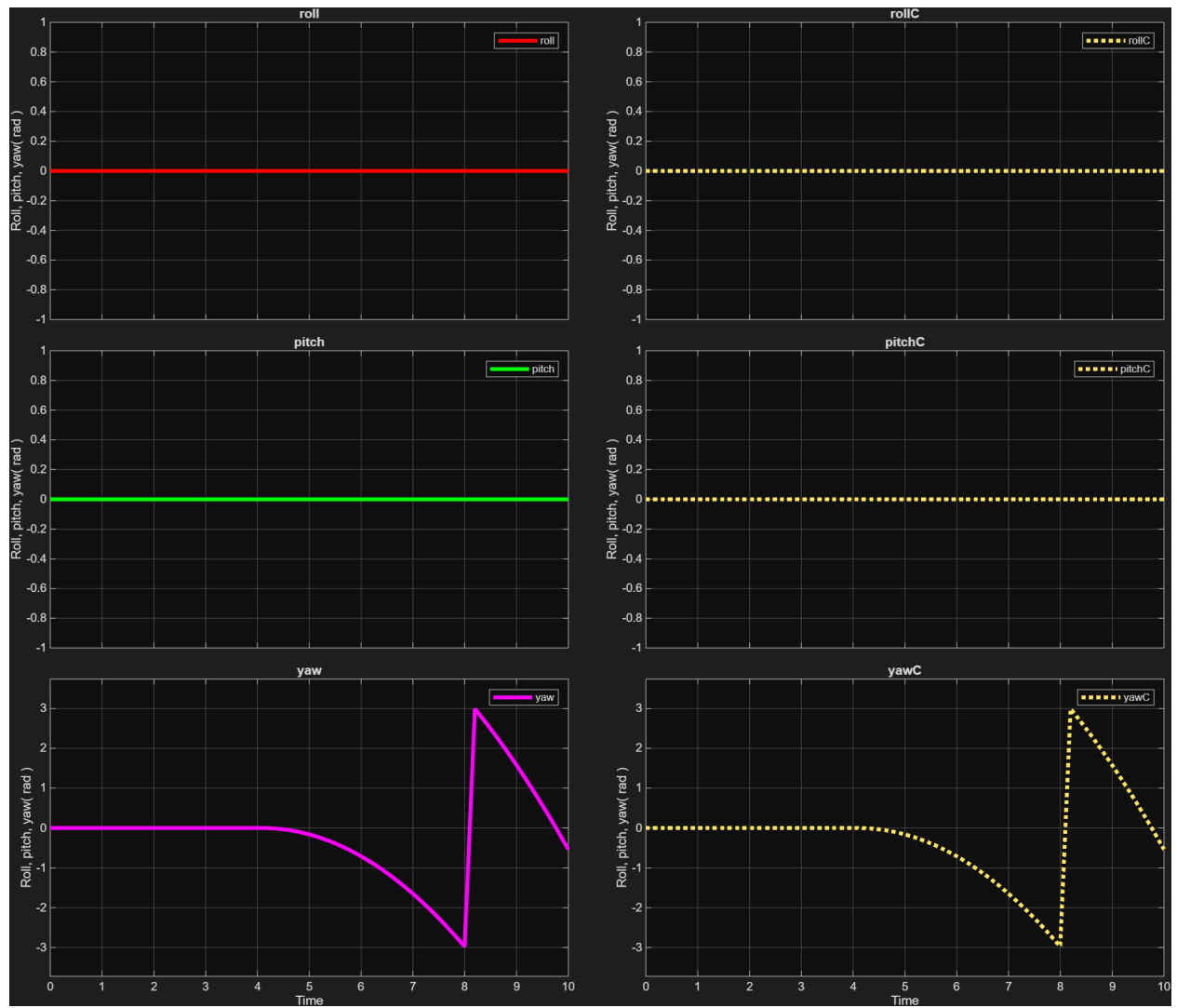


Figure 24. Isolated yaw. When the bigger speed is on rotors 1 and 3.

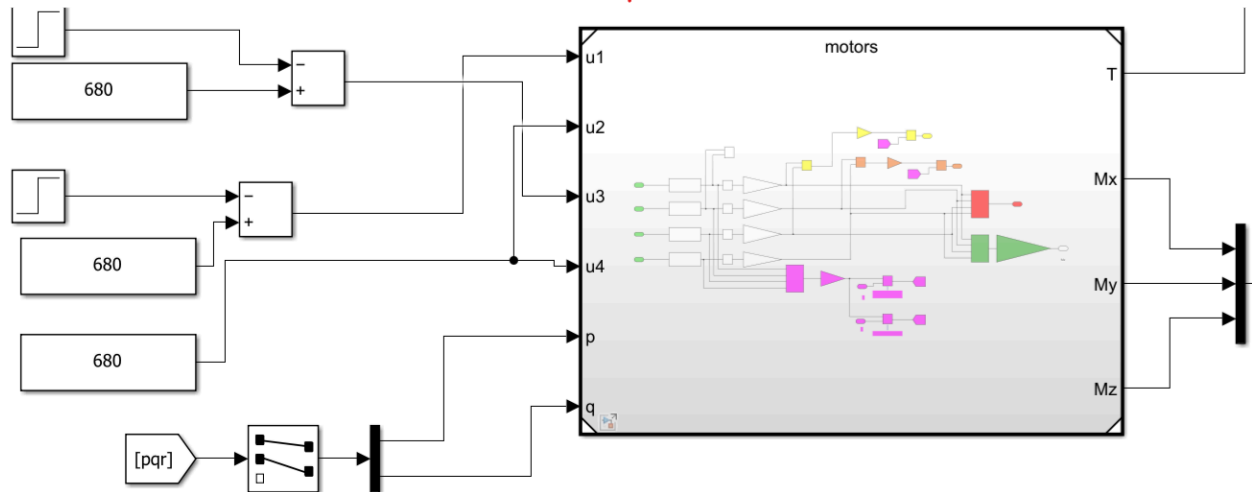


Figure 25. Isolated yaw in simulink model. We use rotors 1 and 3 to control yaw but by reducing speed on them.

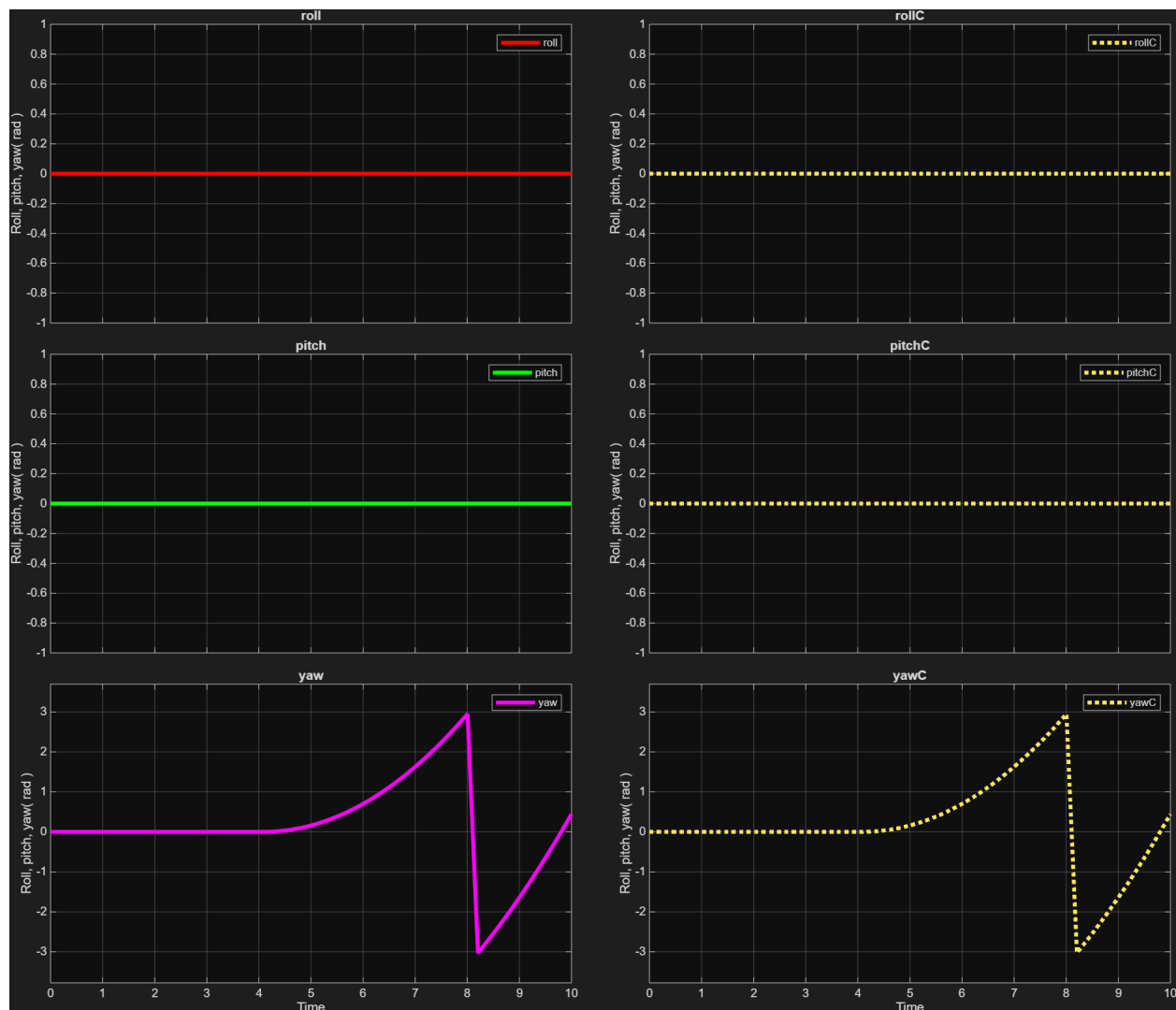


Figure 26. Isolated yaw. When the bigger speed is on rotors 2 and 4 because speed on 1 and 3

was reduced.

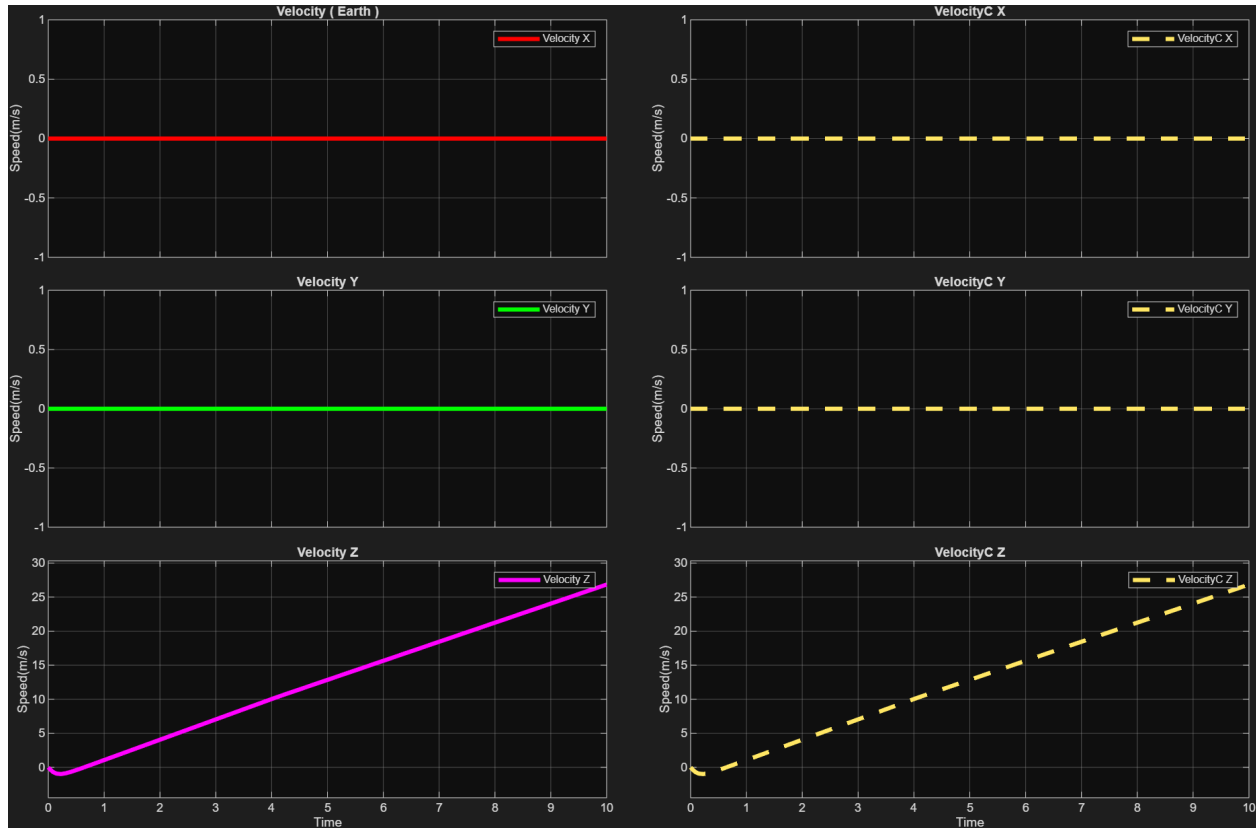


Figure 27. Velocity (Earth frame) during isolated yaw.

Conclusion.

A custom 6-DOF block was developed utilizing a MATLAB Function block, which simplifies the initial design and makes the underlying equations easier to interpret. The custom block fulfills its requirements, and its outputs strictly match the existing Euler-based reference block across all tested scenarios.